

Binary and digital logic

MEGN 441
Zhaoda Du

Upcoming deadlines

- Homework 2 (Gears) due Monday, 09/08
- Lab 1 report due Tuesday, 09/09 (Let me know if you didn't finish the lab)
- Office hours:
 - M 10-12pm
 - If you need to get into lab this week, send me an email!

Data types

```
// Both of these are okay.  
int myVariable = 7;  
unsigned int otherVariable;  
// Once declared, variables can be changed.  
myVariable = 8;  
otherVariable = 9;
```

Keyword	Type	Memory Size	Range
int	Integer	2 bytes (16 bits)	-32,768 to 32,767
long	Long integer	4 bytes (32 bits)	-2,147,483,648 to 2,147,483,647
unsigned int	Unsigned integer	2 bytes	0 to 65,535
unsigned long	Unsigned long	4 bytes	0 to 4,294,967,295
float	Floating point value (7 digits of precision)	4 bytes	-3.4028235E+38 to 3.4028235E+38
char	Character	1 byte (8 bits)	-128 to 127 representing ASCII code
bool	Boolean value	1 byte	0 or 1
byte	Single byte (unsigned)	1 byte	0 to 255

Length of an array

- Arduino C does not have a built-in length function
- However, the **sizeof** function will give you the size of an array in bytes
- An int, for example is 2 bytes, so to iterate through all entries of an array, use:

```
int array[]={1, 2, 3}  
for (int i=0; i < sizeof(array) / 2; i++)  
{  
  \\do something  
}
```

Binary representation of numbers

- A number with n digits can be represented as a sum
 - $364 = 3 \cdot 10^2 + 6 \cdot 10^1 + 4 \cdot 10^0$ decimal notation
- In binary, we use base 2
 - $(13)_{10} = 8 + 4 + 1$
 $= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (1101)_2$
- Use '0b' in C. (represent this is a binary notation)

```
uint a=13  
uint b=0b1101
```

0	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---

Why 8 numbers?

Convert 45 to binary representation

1. $45 \div 2 = 22 \text{ } 1$

2. $22 \div 2 = 11 \text{ } 0$

3. $11 \div 2 = 5 \text{ } 1$

4. $5 \div 2 = 2 \text{ } 1$

5. $2 \div 2 = 1 \text{ } 0$

6. $1 \div 2 = 0 \text{ } 1$

Convert binary to decimal representation

$$(101101)_2 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

- $1 \times 2^5 = 32$
- $0 \times 2^4 = 0$
- $1 \times 2^3 = 8$
- $1 \times 2^2 = 4$
- $0 \times 2^1 = 0$
- $1 \times 2^0 = 1$

$$32 + 0 + 8 + 4 + 0 + 1 = 45$$

$$(101011)_2 = ?$$

43

$$(85)_{10} = ?$$

1010101

Hexadecimal representation

- Base 16
- Use '0x' as prefix in C
- Digits range from 0-9 and then A-F
- 1 digit represents 4 bits in binary $2^4 = 16$
- $0x3B = 3 * 16^1 + 11 * 16^0 = 48 + 11 = 59$

$$0x2F = 2 \times 16^1 + 15 \times 16^0$$

(156)₁₀ to hexadecimal

1. $156 \div 16 = 9 \text{ } 12$

2. $9 \div 16 = 0 \text{ } 9$

$$156_{10} = 0x9C$$

Convert 0x7A to decimal

$$200_{10} = 0xC8$$

Two's complement representation

- Way to represent signed integers in binary
- Let the first bit be negative
$$-c_n(2^n) + c_{n-1}(2^{n-1}) + \dots + c_1(2^1) + c_0(2^0)$$

Decimal	3 Bit	4 Bit
7		0111
6		0110
5		0101
4		0100
3	011	0011
2	010	0010
1	001	0001
0	000	0000
-1	111	1111
-2	110	1110
-3	101	1101
-4	100	1100
-5		1011
-6		1010
-7		1001
-8		1000

Two's complement calculations

- Converting signed binary value to decimal
 - If the first digit is 0, the number is positive and can be used 'as-is'
 - If the first digit is 1, the number is negative
 - Calculate the positive magnitude by subtracting 1, then invert all the bits
 - $-2 = 1110$ $1110 - 1 = 1101$ Invert = $0010 = 2$
- Converting decimal to signed binary
 - Do the above steps in reverse: take magnitude as binary, invert the bits, add 1
 - $-13 \rightarrow 13 = 00001101$ invert = 11110010 add 1 = 11110011
 - $-13 = 0b11110011$ (need at least 5 bits to represent this value)

Practice

- What is -25 in 6-bit signed binary?

- $25 \div 2 = 12 \text{ } 1$
- $12 \div 2 = 6 \text{ } 0$
- $6 \div 2 = 3 \text{ } 0$
- $3 \div 2 = 1 \text{ } 1$
- $1 \div 2 = 0 \text{ } 1$

$$25_{10} = 00011001_2$$

$$11100110$$

$$11100110 + 1 = 11100111$$

- What is 0b101100 as an integer?

$$\begin{aligned} 0b101100 &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 \\ &= 32 + 0 + 8 + 4 + 0 + 0 = 44 \end{aligned}$$

Overflow

- If you add beyond the variable's range, you will see the value jump (127 -> -128 or 255 -> 0)
- Relevant Example
 - Arduino millis(): returns milliseconds as “unsigned long”
 - Max is $\sim 4 \cdot 10^9$ milliseconds, this is about 49.7 days
 - `previous_time - millis()` will produce erroneous results

Floating Point Variable Representation

- Summation Representation

- Decimal: $532.76 = 5*10^2 + 3*10^1 + 2*10^0 + 7*10^{-1} + 6*10^{-2}$
- Binary: $101.01 = 1*2^2 + 0*2^1 + 1*2^0 + 0*2^{-1} + 1*2^{-2} = 5.25$ (decimal)

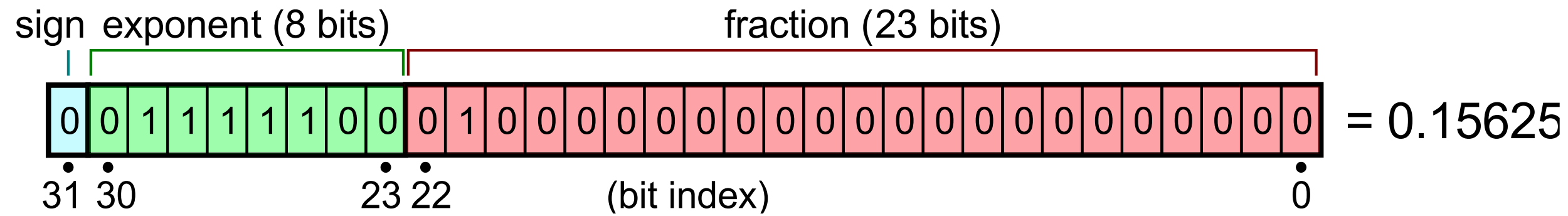
- Scientific Notation

- Decimal: $532.76 = 5.3276*10^2$ (5.3276 = mantissa (significand), 2 = exponent)
- Binary: $101.01 = 1.0101*2^2$ (1.0101 = mantissa, 2 = exponent)

Floating Point Variable Representation

- IEEE 754 Standard (32 bit float): 1-bit sign, 8-bit exponent, 23-bit mantissa
 - Sign: 0 = +, 1 = -
 - Exponent: Unsigned 8-bit value – 127
 - Mantissa: bit left of the binary point assumed 1, 23 bits represent bits right of the binary point
- Digits of Precision
 - $10^d = 2^{24} \rightarrow \log_{10}(2^{24}) = d \approx 7.225$ digits

Floating Point Variable Representation



https://en.wikipedia.org/wiki/Floating-point_arithmetic

- Sign = 0, positive number
- Exponent = $0b01111100 - 127 = 124 - 127 = -3$
- Mantissa (first bit assumed 1) = $1.010000000000000000000000000000$
- $+1.010000000000000000000000000000 * 2^{-3} = (1 * 2^0 + 1 * 2^{-2}) * 2^{-3} = (1 + \frac{1}{4}) * (\frac{1}{8}) = (\frac{5}{4}) * (\frac{1}{8}) = \frac{5}{32} = 0.15625$

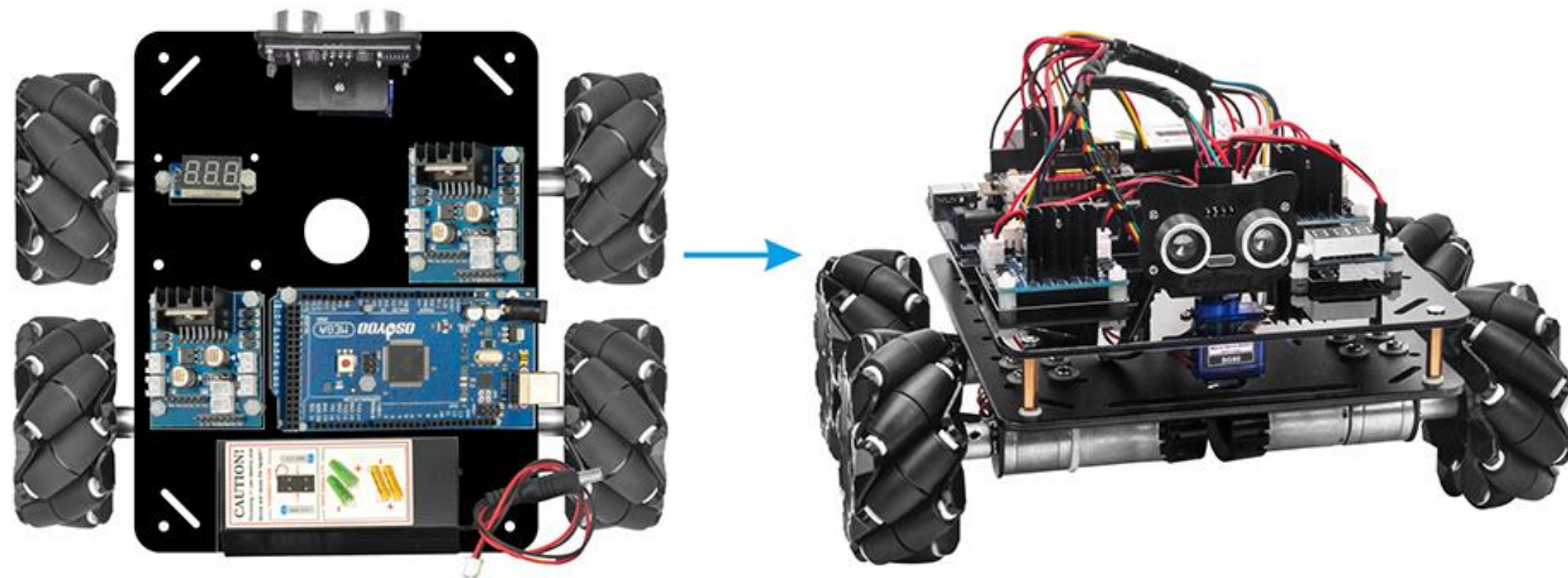
Lab this week

1. Build a car!

We've got step-by-step instructions to build the car in the kit.

2. Start to program motors and test their behavior

3. Week 2 of Lab 2: Attempt to drive a course with open-loop control



L298N Motor Driver

Control each motor with 3 digital pins: IN1, IN2, EN

EN must be a PWM pin

4 motors * 3 pins = 12 pins!

Consider using a structure or array for each motor to keep track of pins.

IN1	IN2	EN	Result
H	L	PWM >0	Clockwise spin
L	H	PWM >0	Counter-clockwise spin
L	L	PWM >0	Fast stop
H	H	PWM >0	Fast stop
Anything	Anything	0	Free-running stop

Arduino folder structure

- Main file
 - Main file.ino
 - Other file.ino
 - Header.h